

Data Structures and Algorithms

Topics:

1. Singly Linked List
2. Doubly Linked list
3. Singly Circular Linked list
4. Arrays
5. Stacks
6. Queues
7. Binary Tree
8. Binary Heaps
9. Graphs
10. Matrix
11. Tries

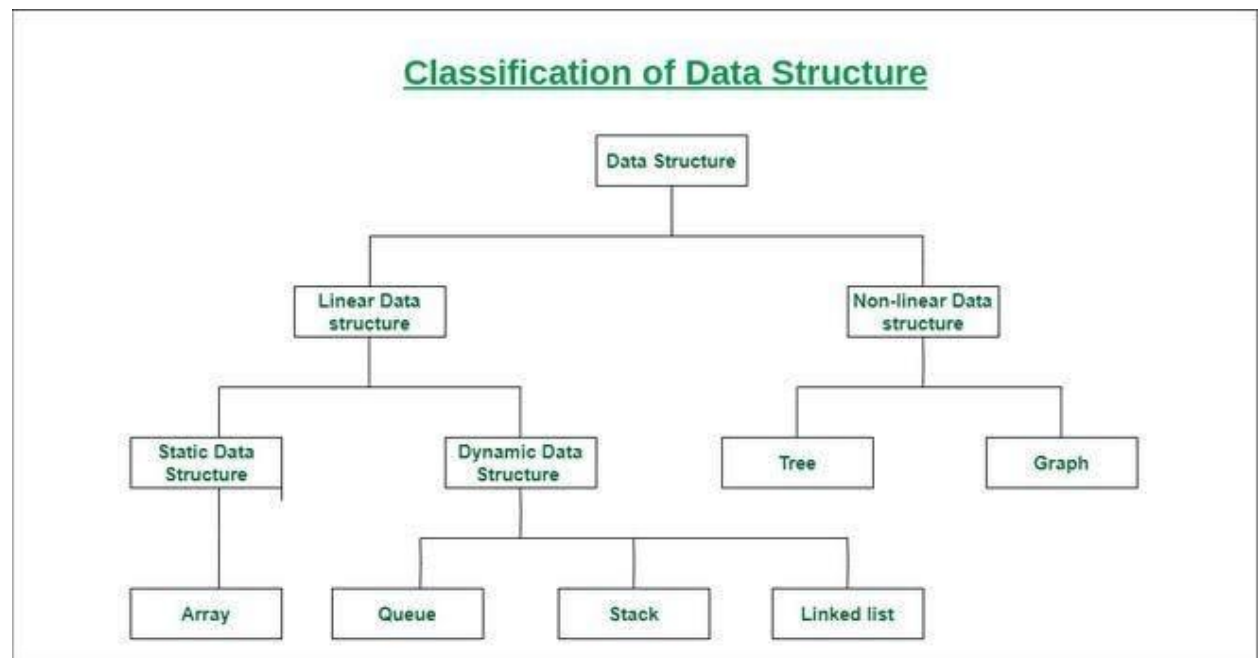
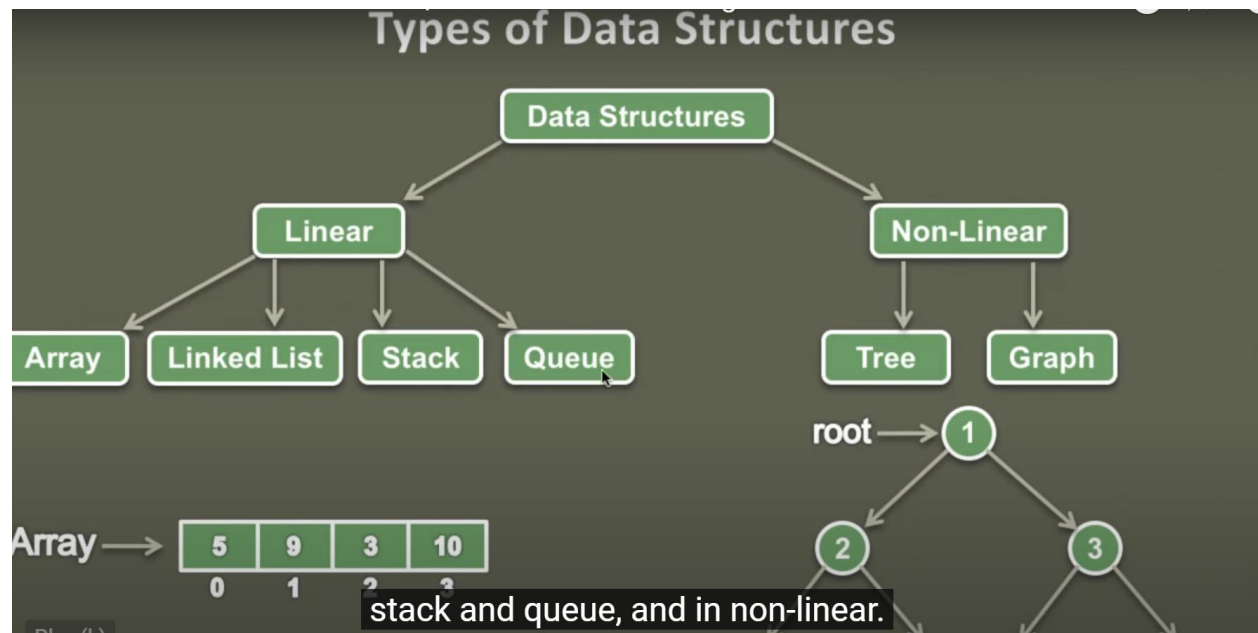
What is Data Structures?

Data Structures

- It is a way to organize data.
- After organizing data it becomes easy to process it.



Types of Data Structures:



Algorithms

- An Algorithm is a set of instructions to perform a task or to solve a given problem.
- For example – A recipe book is a collection of recipes in which each recipe provides an step by step instruction to prepare food.
- Let say you want to prepare a tea. So, the steps would be –

1. Boil water.
2. Put tea in tea pot.
3. Add hot water.
4. Put hot tea into tea cups.
5. Do you need sugar ?
 - a) If yes, put it into tea cups
 - b) If no, do nothing

6. Stir, drink and enjoy!!!

Which help us to solve a given problem.

- Print average of 3 given numbers.
- Let say you want to write algorithm for it. So, the steps would be –

1. Perform sum of 3 numbers.
2. Store it in a variable sum.
3. Divide the sum by 3.
4. Store the value in variable avg.
5. Print the value stored in avg.

```
public void findAvg(int a, int b, int c) {  
    int sum = a + b + c;  
    int avg = sum / 3;  
    System.out.println(avg);  
}
```

Analysis of Algorithms

- An Algorithm is a set of instructions to perform a task or to solve a given problem.
- There are several different algorithms to solve a given problem.
- Analysis of algorithm deals in finding best algorithm which runs fast and takes in less memory.
- For example –

Q. Find sum of first n natural numbers.

a) Input :- n = 4

Output :- 10 i.e. (1 + 2 + 3 + 4)

b) Input :- n = 5

Output :- 15 i.e. (1 + 2 + 3 + 4 + 5)

Analysis of Algorithms

Ramesh

```
public int findSum(int n) {  
    return n * (n + 1) / 2;  
}
```

Suresh

```
public int findSum(int n) {  
    int sum = 0;  
    for(int i = 1; i <= n; i++) {  
        sum = sum + i;  
    }  
    return sum;  
}
```

1. Time Complexity

2. Space Complexity

Time Complexity

- Its amount of time taken by algorithm to run.
- The input processed by an algorithm helps in determining the time complexity.

Ramesh

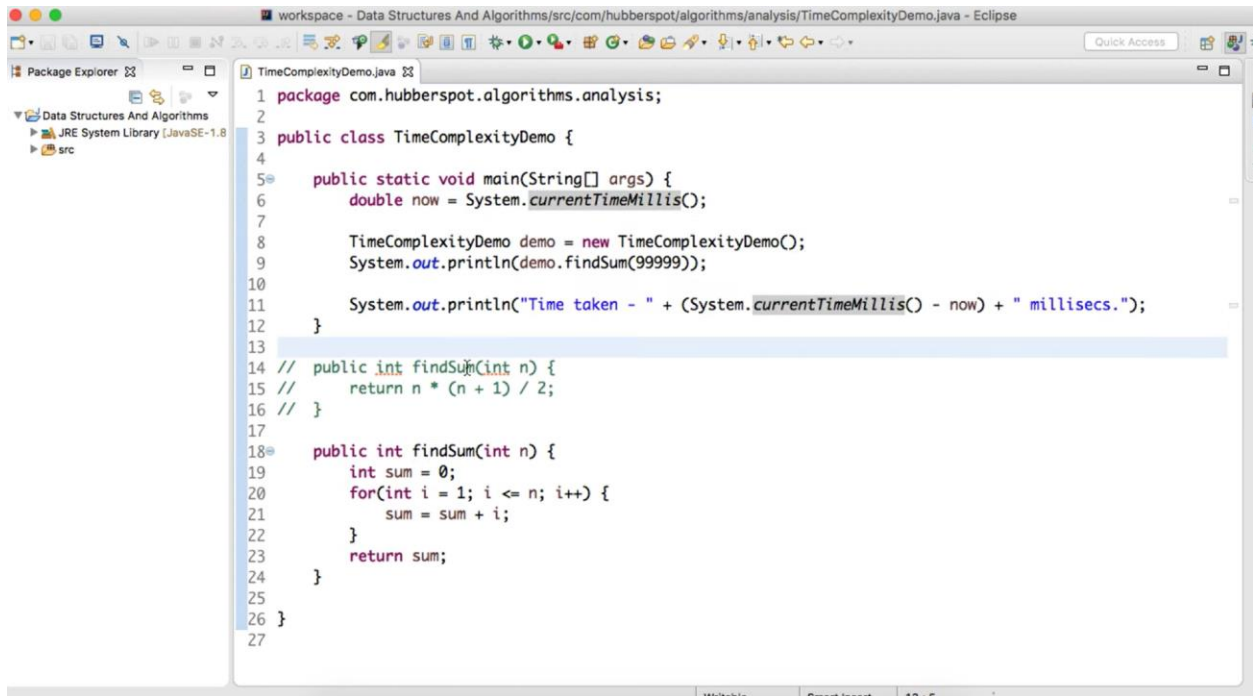
```
public int findSum(int n) {  
    return n * (n + 1) / 2;  
}
```

Suresh

```
public int findSum(int n) {  
    int sum = 0;  
    for(int i = 1; i <= n; i++) {  
        sum = sum + i;  
    }  
    return sum;  
}
```

4 new notifications (Do not disturb)

Example in IDE:



```
1 package com.hubberspot.algorithms.analysis;  
2  
3 public class TimeComplexityDemo {  
4  
5     public static void main(String[] args) {  
6         double now = System.currentTimeMillis();  
7  
8         TimeComplexityDemo demo = new TimeComplexityDemo();  
9         System.out.println(demo.findSum(99999));  
10  
11         System.out.println("Time taken - " + (System.currentTimeMillis() - now) + " millisecs.");  
12     }  
13  
14     // public int findSum(int n) {  
15     //     return n * (n + 1) / 2;  
16     // }  
17  
18     public int findSum(int n) {  
19         int sum = 0;  
20         for(int i = 1; i <= n; i++) {  
21             sum = sum + i;  
22         }  
23         return sum;  
24     }  
25  
26 }  
27
```

Space Complexity

- Its amount of memory or space taken by algorithm to run.
- The memory required to process the input by an algorithm helps in determining the space complexity.

Ramesh

```
public int findSum(int n) {  
    return n * (n + 1) / 2;  
}
```

Suresh

```
public int findSum(int n) {  
    int sum = 0;  
    for(int i = 1; i <= n; i++) {  
        sum = sum + i;  
    }  
    return sum;  
}
```

There are some mathematical models which determines time and space complexity of and Algorithm:

Asymptotic Analysis of an Algorithm

Asymptotic Analysis

- Asymptotic analysis helps in evaluating performance of an algorithm in terms of input size and its increase.
- Using asymptotic analysis we don't measure actual running time of algorithm.
- It helps in determining how time and space taken by algorithm increases with input size.

Asymptotic Notations:

Asymptotic Notations

- Asymptotic Notations are the mathematical tools used to describe the running time of an algorithm in terms of input size.
- Example – Performance of car in 1 litre of petrol

Highway (min traffic) – 25 km/litre
City (max traffic) – 15 km/litre
City + Highway (avg traffic) – 20 km/litre

Asymptotic Notations help us in determining –

1. Best Case
2. Average Case
3. Worst Case



Types of Asymptotic Notations

- There are three notations for performing runtime analysis of an algorithm -
 1. Omega(Ω) Notation
 2. Big O (O) Notation
 3. Theta (Θ) Notation

1 Omega Notation: (Minimum time /lower bound) – Best Case

Omega(Ω) Notation

- It is the formal way to express the lower bound of an algorithm's running time.
- Lower bound means for any given input this notation determines best amount of time an algorithm can take to complete.
- For example –
If we say certain algorithm takes 100 secs as best amount of time. So, 100 secs will be lower bound of that algorithm. The algorithm can take more than 100 secs but it will not take less than 100 secs.

2. Big O notation: (Worst case) Maximum time

Big O(O) Notation

- It is the formal way to express the upper bound of an algorithm's running time.
- Upper bound means for any given input this notation determines longest amount of time an algorithm can take to complete.
- For example –
If we say certain algorithm takes 100 secs as longest amount of time. So, 100 secs will be upper bound of that algorithm. The algorithm can take less than 100 secs but it will not take more than 100 secs.

3. Theta Notation: (Average Case)

Theta(Θ) Notation

- It is the formal way to express both the upper and lower bound of an algorithm's running time.
- By Lower and Upper bound means for any given input this notation determines average amount of time an algorithm can take to complete.
- For example –
If we run certain algorithm and it takes 100 secs for first run, 120 secs for second run, 110 for third run and so on. So, theta notations gives an average of running time of that algorithm.

Analysis of Time Complexity (Big O Notation):

Big O(O) Notation

- It is the formal way to express the upper bound of an algorithm's running time.
- Upper bound means for any given input this notation determines longest amount of time an algorithm can take to complete.

Rules of Big O Notation:

Rules of Big O(O) Notation

- It's a Single Processor
- It performs Sequential Execution of statements
- Assignment operation takes 1 unit of time
- Return statement takes in 1 unit of time
- Arithmetical operation takes 1 unit of time
- Logical operation takes 1 unit of time
- Other small/single operations takes 1 unit of time
- Drop lower order terms



$$T = n^2 + 3n + 1 \implies O(n^2)$$

- Drop lower order terms
- Drop constant multipliers



$$T = 3n^2 + 6n + 1 \implies O(n^2)$$

Calculating Time Complexity of Constant Algorithm:

```

1 public int sum(int x, int y) {
2     int result = x + y;
3     return result;
4 }
    
```

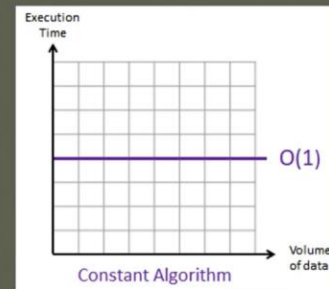
line no.	operations	unit time
2	1 + 1 + 1 + 1	4
3	1 + 1	2

$$T = 4 + 2 = 6$$

$$T \approx C \text{ (constant)}$$

```

public int get(int[] arr, int i) {
    return arr[i];
}
    
```



Time Complexity = $O(1)$

Credit: 101computing.net

Lin

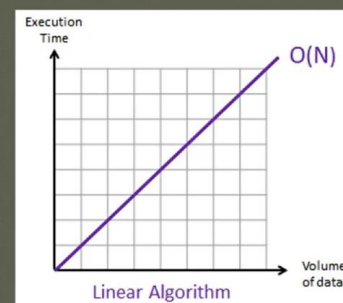
```

1 public void findSum(int n) {
2     int sum = 0; // 1 step
3     for(int i = 1; i <= n; i++) {
4         sum = sum + i; // n steps
5     }
6     return sum; // 1 step
7 }
    
```

line no.	operations	unit time
2	1	1
3	1 + 3n + 3 + 3n	6n + 4
4	n(1 + 1 + 1 + 1)	4n
6	1 + 1	2

$$T = 1 + 6n + 4 + 4n + 2$$

$$T = \cancel{10n} + \cancel{7}$$



Time Complexity = $O(n)$

Credit: 101computing.net

Polynomial Algorithm(Big O):

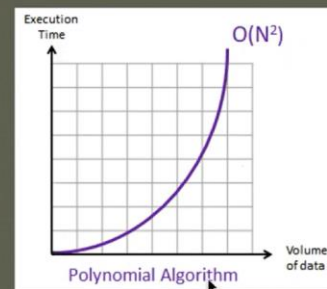
```
1 public void print(int n) {  
2     for(int i = 1; i <= n; i++) {  
3         for(int j = 1; j <= n; j++) {  
4             s.o.p("i = " + i + ", j = " + j);  
5         }  
6         s.o.p("End of inner loop");  
7     }  
8     s.o.p("End of outer loop");  
9 }
```

line no.	operations	unit time
2	$1 + 3n + 3 + 3n$	$6n + 4$
3	$n(1 + 3n + 3 + 3n)$	$6n^2 + 4n$
4	$n^2(1 + 1 + 1)$	$3n^2$
6	$n(1)$	n
8	1	1

$$T = 6n + 4 + 6n^2 + 4n + 3n^2 + n + 1$$

$$T = \cancel{9n^2} + \cancel{11n} + \cancel{5}$$

Time Complexity = $O(n^2)$



Array Data Structure:

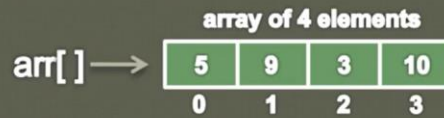
What is an Array?

Chocolate box



1. It's a collection (box) of chocolates.
2. All chocolate holding partitions are adjacent or contiguous.
3. Each partition has two neighbors except first and last one.
4. Size of box is fixed and cannot be modified.
5. Being adjacent each partition is indexed and can be determined by its position.

What is an Array?



1. It's a collection (box) of data elements of specified type.
2. All data holding partitions have contiguous memory locations.
3. Each partition has two neighbors except first and last one.
4. Size of array is fixed and cannot be modified once it is created.
5. Being adjacent each partition is indexed and can be determined by its position.
6. Index starts at 0 and for (one dimensional array) ends at length - 1

Why array index starts from 0 in java?

Reference Links:

<https://medium.com/@varshilshah.in/why-array-indexing-starts-from-zero-11572281920>

<https://medium.com/@piyush.kochhar1/why-array-index-begins-with-0-9adfef92223a>

Array indices in Java (and many other programming languages) start from 0 due to historical and practical reasons rooted in computer science. Here are the main reasons:

1. ****Historical Context****:

- The tradition of zero-based indexing comes from early programming languages like C. C's array indexing directly corresponds to memory addresses, where the index serves as an offset from the starting memory address. The first element is at the starting address plus zero, hence index 0.

2. ****Pointer Arithmetic****:

- In languages like C and C++, arrays are closely tied to pointer arithmetic. For example, if an array starts at memory address `p`, the element at index `i` is located at `p + i \* sizeof(element)`. Starting the index at 0 means the first element is exactly at the address `p`, simplifying this calculation.

3. ****Simplicity and Efficiency****:

- Zero-based indexing can simplify the implementation of arrays and reduce the chance of off-by-one errors in calculations. It aligns with the way memory addressing works, thus allowing more straightforward and efficient computation.

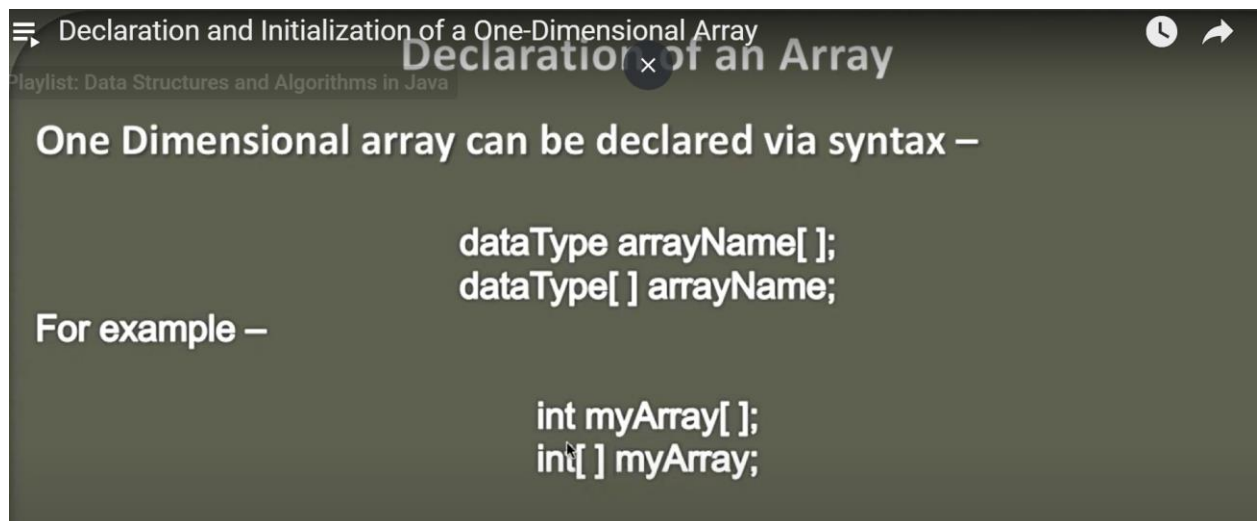
4. ****Consistency Across Languages****:

- Many modern programming languages (such as Java, Python, and JavaScript) have adopted zero-based indexing for consistency with C and other early languages. This makes it easier for developers to switch between languages without having to adjust to different indexing conventions.

5. ****Mathematical Convenience****:

- In many mathematical contexts, zero-based indexing can be more convenient. For instance, in modulo arithmetic and many algorithms, starting from 0 can simplify calculations and loop conditions.

These reasons combine to make zero-based indexing a standard choice for arrays in Java and many other programming languages.



The screenshot shows a video player interface. At the top, there is a title bar with a hamburger menu icon, the text 'Declaration and Initialization of a One-Dimensional Array', a close button (X), and a share icon. Below the title bar, a subtitle reads 'Playlist: Data Structures and Algorithms in Java'. The main content area of the video is a dark green slide with white text. The slide title is 'Declaration of an Array'. The main text on the slide says 'One Dimensional array can be declared via syntax –' followed by two lines of Java syntax: `dataType arrayName[ ];` and `dataType[ ] arrayName;`. Below this, it says 'For example –' followed by two lines of example code: `int myArray[ ];` and `int[ ] myArray;`.

Declaration and Initialization of a One-Dimensional Array

Playlist: Data Structures and Algorithms in Java

Declaration of an Array

One Dimensional array can be declared via syntax –

```
dataType arrayName[ ];
dataType[ ] arrayName;
```

For example –

```
int myArray[ ];
int[ ] myArray;
```

Declaration and Initialization

The array can be declared and initialized in same line via syntax-

```
dataType[ ] arrayName = new dataType[size];  
dataType arrayName[ ] = new dataType[size];
```

For example –

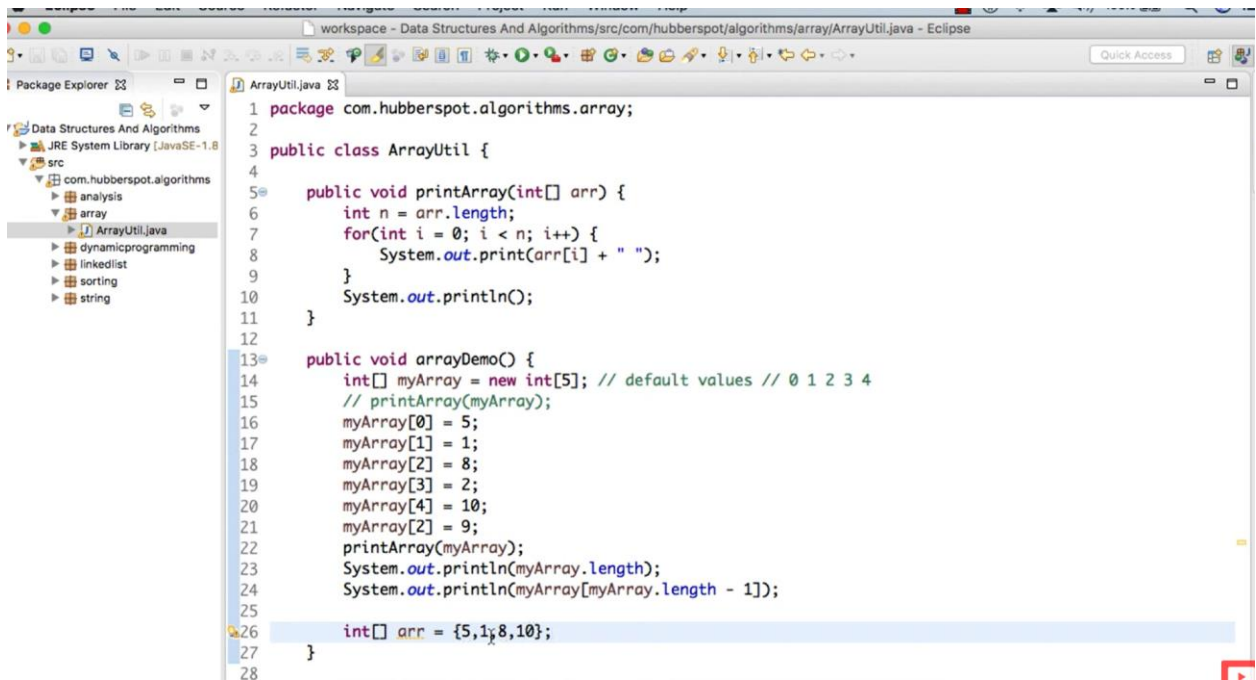
```
int[ ] myArray = new int[5];  
int myArray[ ] = new int[5];  
int[ ] myArray = {5,4,3,2,6};
```

Adding or Updating elements in an Array

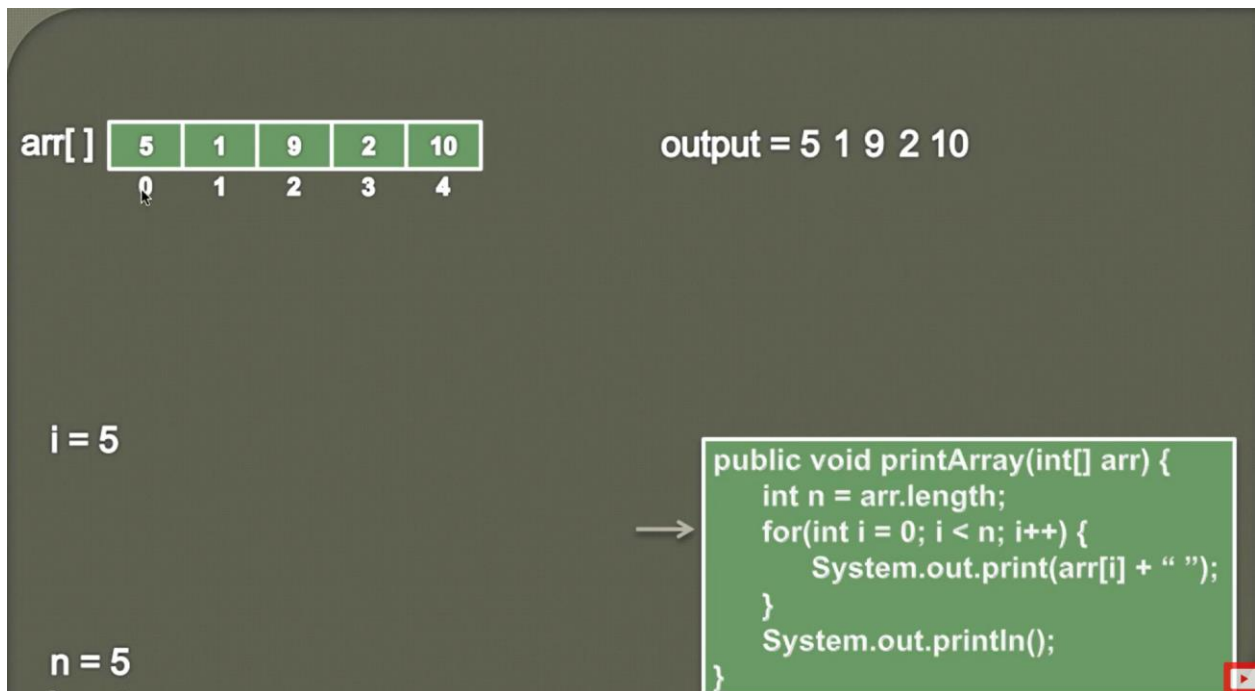


java.lang.ArrayIndexOutOfBoundsException:
Index 5 out of bounds for length 5

```
public void arrayDemo() {  
    int[ ] myArray = new int[5];  
    myArray[0] = 5;  
    myArray[1] = 1;  
    myArray[2] = 8;  
    myArray[3] = 2;  
    myArray[4] = 10;  
    myArray[2] = 9;  
    myArray[5] = 7;  
}
```



```
1 package com.hubberspot.algorithms.array;
2
3 public class ArrayUtil {
4
5     public void printArray(int[] arr) {
6         int n = arr.length;
7         for(int i = 0; i < n; i++) {
8             System.out.print(arr[i] + " ");
9         }
10        System.out.println();
11    }
12
13    public void arrayDemo() {
14        int[] myArray = new int[5]; // default values // 0 1 2 3 4
15        // printArray(myArray);
16        myArray[0] = 5;
17        myArray[1] = 1;
18        myArray[2] = 8;
19        myArray[3] = 2;
20        myArray[4] = 10;
21        myArray[2] = 9;
22        printArray(myArray);
23        System.out.println(myArray.length);
24        System.out.println(myArray[myArray.length - 1]);
25    }
26    int[] arr = {5,1,8,10};
27 }
28
```



The screenshot shows the Eclipse IDE workspace. The Package Explorer on the left shows the project structure: Data Structures And Algorithms > JRE System Library [JavaSE-1.8] > src > com.hubberspot.algorithms > array > dynamicprogramming. The main editor shows the file ArrayUtil.java with the following code:

```

1 package com.hubberspot.algorithms.array;
2
3 public class ArrayUtil {
4
5     public void printArray(int[] arr) {
6         int n = arr.length;
7         for(int i = 0; i < n; i++) {
8             System.out.print(arr[i] + " ");
9         }
10        System.out.println();
11    }
12
13    public static void main(String[] args) {
14        ArrayUtil arrUtil = new ArrayUtil();
15        arrUtil.printArray(new int[] {5, 1, 2, 9, 10});
16    }
17 }
18
19

```

The Console on the right shows the output of the program:

```

<terminated> ArrayUtil [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_111.jdk/Contents/Home/
5 1 2 9 10

```

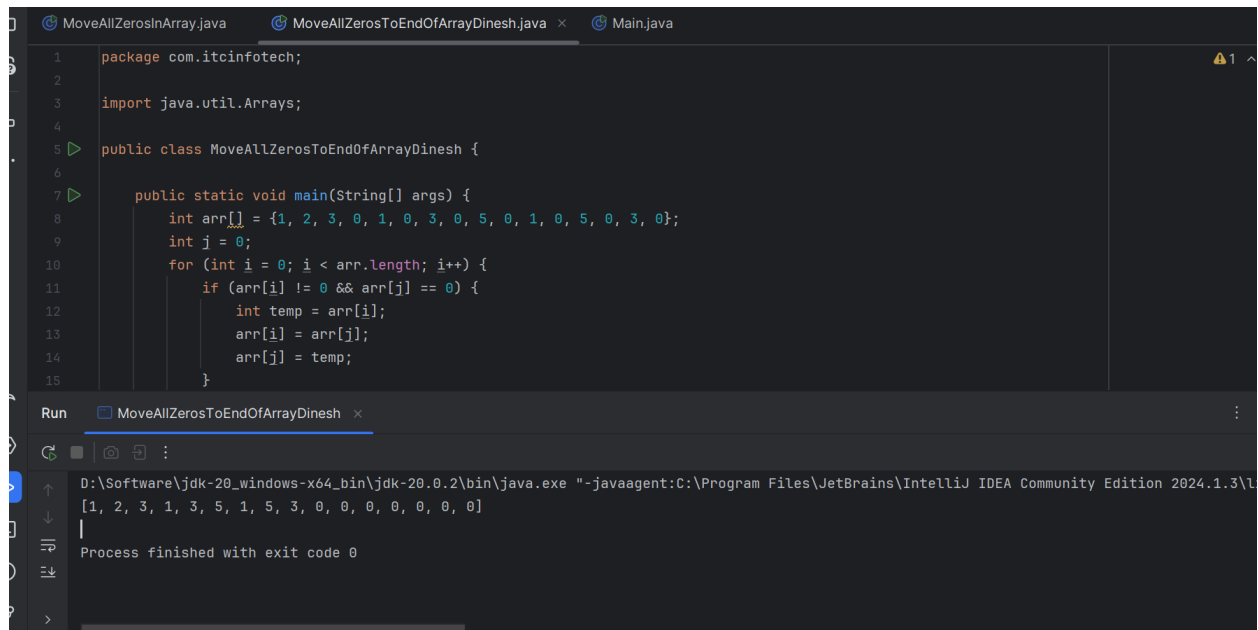
The diagram illustrates the logic of the `removeEven` method. It shows an input array `arr[]` with values [3, 2, 4, 7, 10, 6, 5] and indices [0, 1, 2, 3, 4, 5, 6]. An arrow labeled `i` points to the element at index 6 (value 5). Below it, the `result[]` array is shown with values [3, 7, 5] and indices [0, 1, 2]. The text `oddCount = 3` and `removeEven(arr);` is displayed. To the right, the code for the `removeEven` method is shown:

```

int[] removeEven(int[] arr) {
    int oddCount = 0;
    for(int i = 0; i < arr.length; i++){
        if(arr[i] % 2 != 0){
            oddCount++;
        }
    }
    int[] result = new int[oddCount];
    int idx = 0;
    for(int i = 0; i < arr.length; i++){
        if(arr[i] % 2 != 0){
            result[idx] = arr[i];
            idx++;
        }
    }
    return result;
}

```

1. How to find second mx and second min value from given array?
2. How to move all zeros to the end of array? By maintaining relative order of non-zero values?



The screenshot shows the IntelliJ IDEA IDE with three tabs: `MoveAllZerosInArray.java`, `MoveAllZerosToEndOfArrayDinesh.java`, and `Main.java`. The `MoveAllZerosToEndOfArrayDinesh.java` tab is active, displaying the following Java code:

```
1 package com.itcinfotech;
2
3 import java.util.Arrays;
4
5 public class MoveAllZerosToEndOfArrayDinesh {
6
7     public static void main(String[] args) {
8         int arr[] = {1, 2, 3, 0, 1, 0, 3, 0, 5, 0, 1, 0, 5, 0, 3, 0};
9         int j = 0;
10        for (int i = 0; i < arr.length; i++) {
11            if (arr[i] != 0 && arr[j] == 0) {
12                int temp = arr[i];
13                arr[i] = arr[j];
14                arr[j] = temp;
15            }
16        }
17    }
18 }
```

Below the code editor, the `Run` tab is visible, showing the execution command and output:

```
D:\Software\jdk-20_windows-x64_bin\jdk-20.0.2\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.1.3\l
[1, 2, 3, 1, 3, 5, 1, 5, 3, 0, 0, 0, 0, 0, 0, 0]
Process finished with exit code 0
```

Videos number – 31

Q) How to re-size of array ?

Important Notes:

1. When you try to change the reference of parameter/another method stack variable in another method stack area with another local variable which is created inside the called method area, it will not impact the parameterized variable reference. After method execution completes, all the method local variable sand assignments will not not have considered and it will be eligible for garbage collector.
2. What we should understand like, one method stack areas references will not impact another method stack created reference variables. For below examples – **arr** and **temp** array variables.
3. Here **line number 25**, we assigned **arr = temp**, still we are not seeing those changes in main method place as `resize()` method will not impact the the other method stack area variables.

```
MoveAllZerosInArray.java  MoveAllZerosToEndOfArrayDinesh.java  ResizeArrayInJava.java  Main.java
5 public class ResizeArrayInJava {
6     public static void main(String[] args) {
9         int[] response = resize(arr, capacity: 20);
10        System.out.println(response.hashCode()); //20
11        System.out.println(Arrays.toString(response)); //20
12        System.out.println(arr.length); //9
13        System.out.println(Arrays.toString(arr)); //9
14        System.out.println(arr.hashCode()); //9
15    }
16
17    public static int[] resize(int[] arr, int capacity) { 1 usage
18        System.out.println("arr inside resize method hashCode" + arr.hashCode()); //9
19        arr[0] = 100000;
20        int[] temp = new int[capacity];
21        for(int i=0; i<arr.length; i++) {
22            temp[i] = arr[i];
23        }
24        System.out.println("temp inside resize method hashCode" + temp.hashCode()); //9
25        arr = temp;
26        System.out.println("arr " + Arrays.toString(arr)); //20
27        System.out.println("temp array" + Arrays.toString(temp)); //9
28        arr[1] = 99999;
29        System.out.println("arr inside after assigning with temp resize method hashCode" + arr.hashCode()); //9
30    }
31}
```

Q. Given an array of $n - 1$ distinct numbers in the range of 1 to n . Find the missing number in it.

arr[]

2	4	1	8	6	3	7
0	1	2	3	4	5	6

 7 numbers

Output: 5

Explanation: The missing number from range 1 to 8 is 5.

```
public static void main(String[] args) {

    int[] arr = {1, 4, 6, 7, 3, 9, 5, 2};
    int j = 0;
    //Arrays.sort(arr);
    sortArray(arr);
    for (int i = 0; i < arr.length; i++) {
        int current = arr[i];
        int expectedValue = current + 1;
        if (expectedValue != arr[++i]) {
            System.out.println("Missing value in teh array series====>" + expectedValue);
            break;
        }
    }
    System.out.println(Arrays.toString(arr));
}
```

Mathematical Approach

The sum of first n natural numbers = $1 + 2 + 3 + 4 \dots + n = \underline{n \times (n + 1) / 2}$

arr[]	1	2	3	5
	0	1	2	3

4 numbers
1 to 5

$$\frac{5 \times (5 + 1)}{2} = \frac{30}{2} = 15$$

Output: 4

Explanation: The missing number from range 1 to 5 is 4.

https://www.youtube.com/watch?v=Q485CWk_QWk&list=PL6Zs6LgrJj3tDXv8a_elC6eT_4R5gfX4d&index=34

https://www.youtube.com/watch?v=Q485CWk_QWk&list=PL6Zs6LgrJj3tDXv8a_elC6eT_4R5gfX4d&index=34

```
java  PalindromeDemo.java x  PalindromeDemo1.java  PalindromeUsingRecursive.java  java-class-path-demo\...\Main.java  FindMissingNum:  v
3  public class PalindromeDemo {
4      public static void main(String[] args) {
10         // Time Complexity of palindrome = O(n/2)
11     }
12
13  @  public static boolean isPalindrome(String word) { 3 usages
14      char[] charArray = word.toCharArray();
15      int start = 0;
16      int end = charArray.length - 1;
17
18      while (start < end) {
19          if(charArray[start] != charArray[end]) {
20              return false;
21          }
22          start++;
23          end--;
24      }
25      return true;
26  }
27  }
28  }
```

26th June 2024:

Represent Singly Linked List in Java:

What is Node?: Node is a network component or a device that is part of a larger network. It can be a computer, a printer, a router, or any other electronic device that is connected to a network. The term "node" stands for "Network Open Distributed Environment" and it represents a specific location in a network.

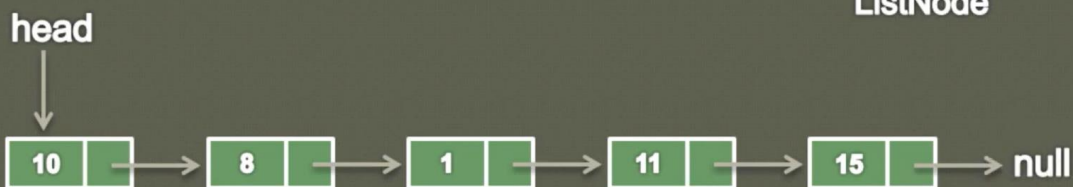
NODE - "Network Open Distributed Environment"

<https://www.linkedin.com/pulse/node-full-form-justacademy-pune-hlmrc/>

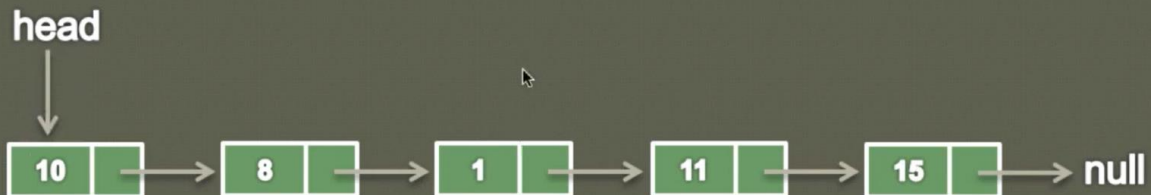
Singly Linked List

Singly Linked List is a data structure used for storing collection of nodes and has following properties –

- It contains sequence of nodes.
- A node has data and reference to next node in a list.
- First node is the head node.
- Last node has data and points to null.

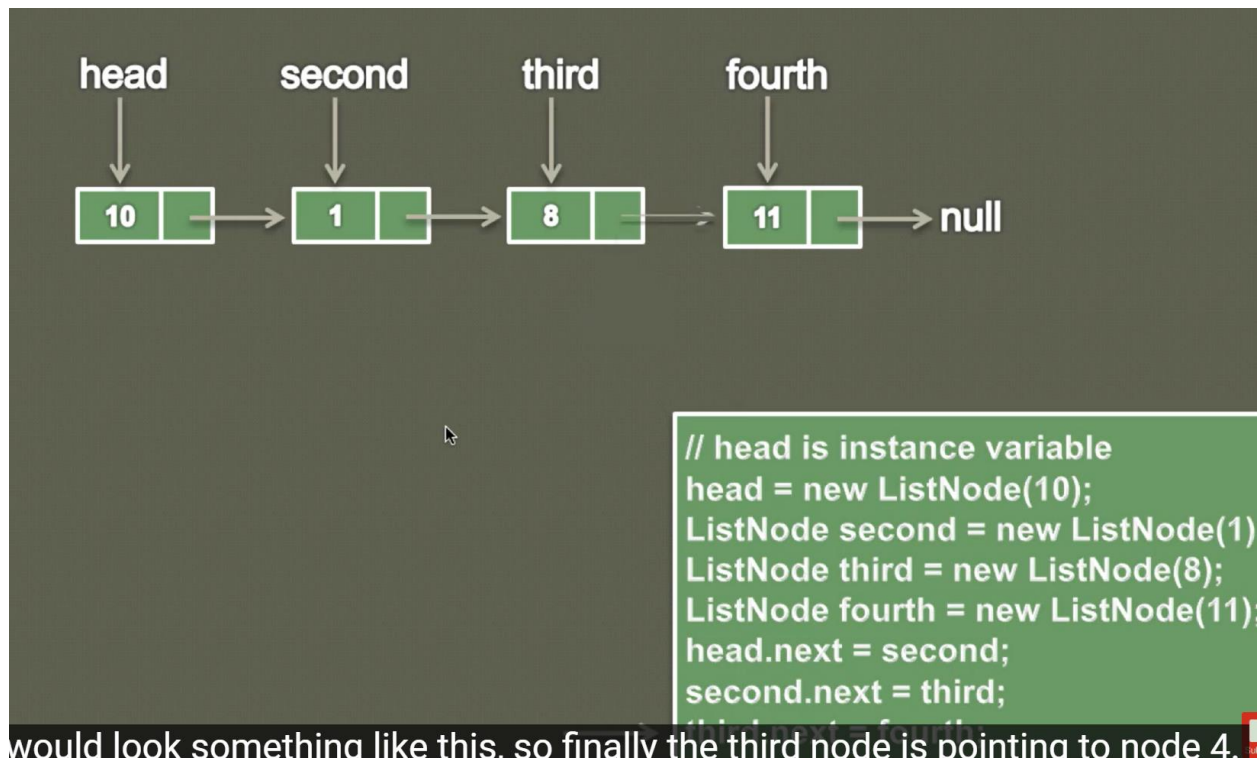


Representation of a Singly Linked List



```
SinglyLinkedList.java
1 public class SinglyLinkedList {
2
3     private ListNode head;
4
5     private static class ListNode {
6         private int data; // Generic Type
7         private ListNode next;
8
9         public ListNode(int data) {
10             this.data = data;
11             this.next = null;
12         }
13     }
14
15 }
```

Videos – 38 - How to create single linked list in java:



would look something like this. so finally the third node is pointing to node 4.

Pass by value vs Pass by reference:

Pass by value in Java: Java is a pass-by-value language in both cases — primitive data types as well as objects. When passing objects to methods, it's important to remember that their reference values (memory addresses) are being passed by value

Example 1: Primitive Data Types

```
public class PassByValueDemo {

    public static void main(String[] args) {
        int number = 10;
        System.out.println("Before: " + number); // Output: Before: 10
        modifyNumber(number);
        System.out.println("After: " + number); // Output: After: 10
    }

    public static void modifyNumber(int value) {
        value = value * 2;
        System.out.println("Modified: " + value); // Output: Modified: 20
    }
}
```

Example 2: Objects

```
public class PassByValueDemo {

    public static void main(String[] args) {
        Point point = new Point(2, 3);
        System.out.println("Before: " + point); // Output: Before: (2, 3)
        modifyPoint(point);
        System.out.println("After: " + point); // Output: After: (20, 30)
    }

    public static void modifyPoint(Point point) {
        point.setX(point.getX() * 10);
        point.setY(point.getY() * 10);
        System.out.println("Modified: " + point); // Output: Modified: (20, 30)
    }
}

class Point {
    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() {
        return x;
    }

    public void setX(int x) {
        this.x = x;
    }

    public int getY() {
        return y;
    }

    public void setY(int y) {
        this.y = y;
    }

    @Override
```

```
public String toString() {  
    return "(" + x + ", " + y + ")";  
}  
}
```

Reference links:

- <https://medium.com/javarevisited/is-java-pass-by-reference-or-pass-by-value-f03562367446#:~:text=Pass%2Dby%2Dvalue%3A%20It,t%20affect%20the%20original%20variable.>
- <https://ananya281294.medium.com/java-passing-by-value-or-passing-by-reference-c75e312069ed>

<https://www.geeksforgeeks.org/pass-by-reference-in-c/>

Pass by value: The method parameter values are copied to another variable and then the copied object is passed to the method. The method uses the copy. **Pass by reference:** An alias or reference to the actual parameter is passed to the method. The method accesses the actual parameter.

```
#include <stdio.h>  
  
// function definition with relevant pointers to receive the  
// parameters  
void swap(int* a, int* b)  
{  
    printf(" Before method body : a is %d and b is %d\n", *a,  
        *b);  
    // accessing arguments like pointers  
    //int temp = *a;  
    *a = 15;  
    *b = 20;  
    // *b = temp;  
    printf(" after method : a is %d and b is %d\n", *a,  
        *b);  
}  
  
// driver code  
int main(void)  
{  
    int n1 = 5;
```

```

int n2 = 10;

// value before swapping
printf(" Before swapping : n1 is %d and n2 is %d\n", n1,
      n2);
// calling the function by passing the address of the
// arguments
swap(&n1, &n2);

// value after swapping
printf(" After swapping : n1 is %d and n2 is %d\n", n1,
      n2);

return 0;
}

```

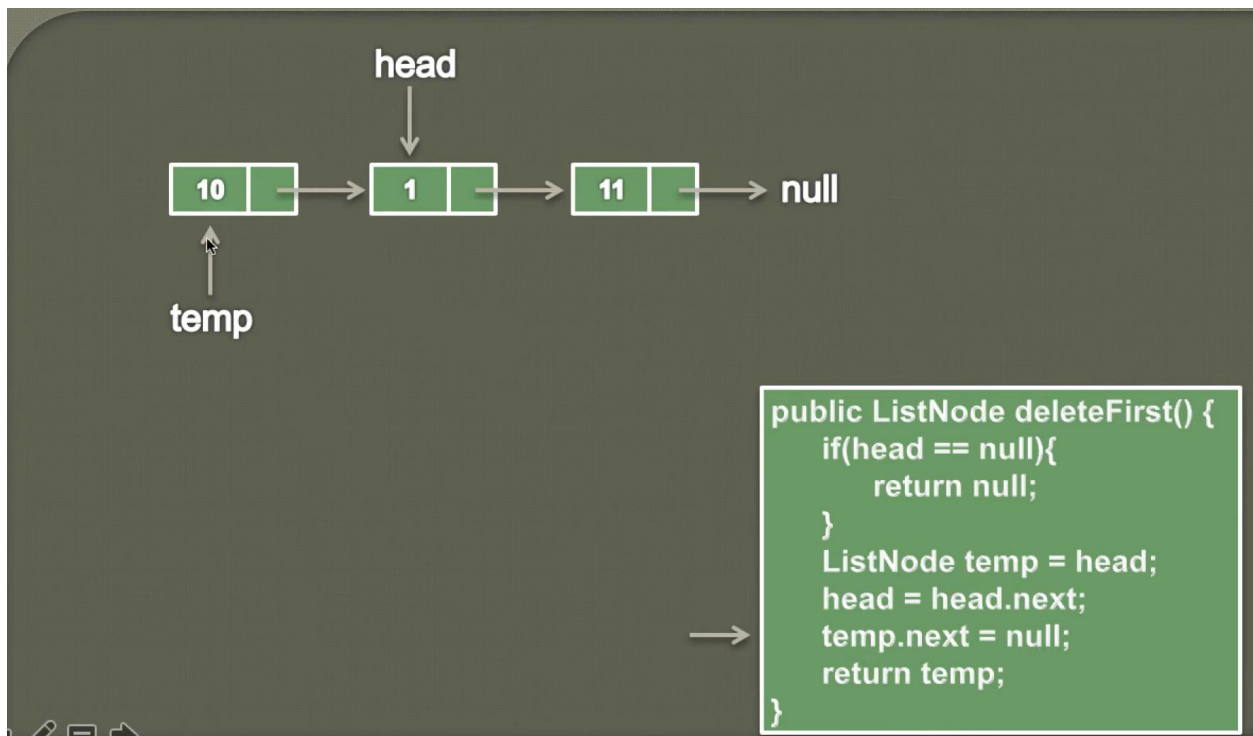
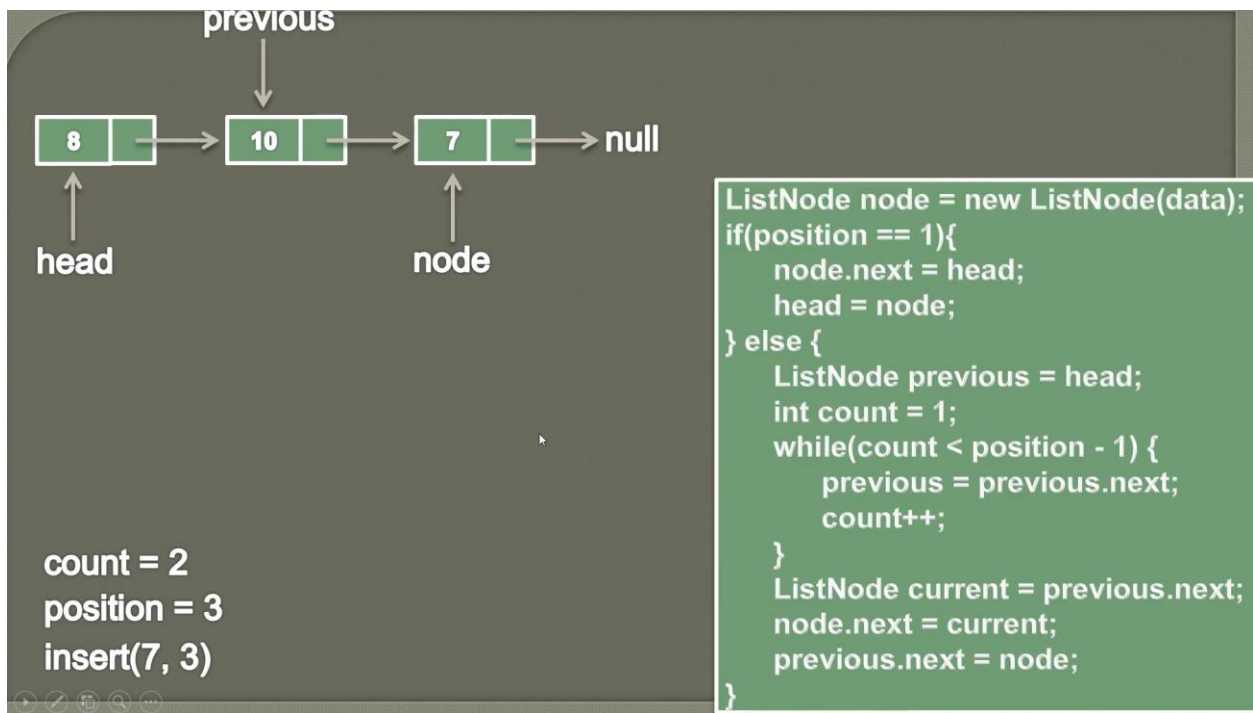
Output:

main.c	Run	Output
<pre> 1 #include <stdio.h> 2 3 // function definition with relevant pointers to recieve the 4 // parameters 5 void swap(int* a, int* b) 6 { 7 printf(" Before method body : a is %d and b is %d\n", *a, 8 *b); 9 // accessing arguments like pointers 10 int temp = *a; 11 *a = 15; 12 *b = 20; 13 //*b = temp; </pre>		<pre> /tmp/G0feFh34EL.o Before swapping : n1 is 5 and n2 is 10 Before method body : a is 5 and b is 10 after method : a is 15 and b is 20 After swapping : n1 is 15 and n2 is 20 === Code Execution Successful === </pre>

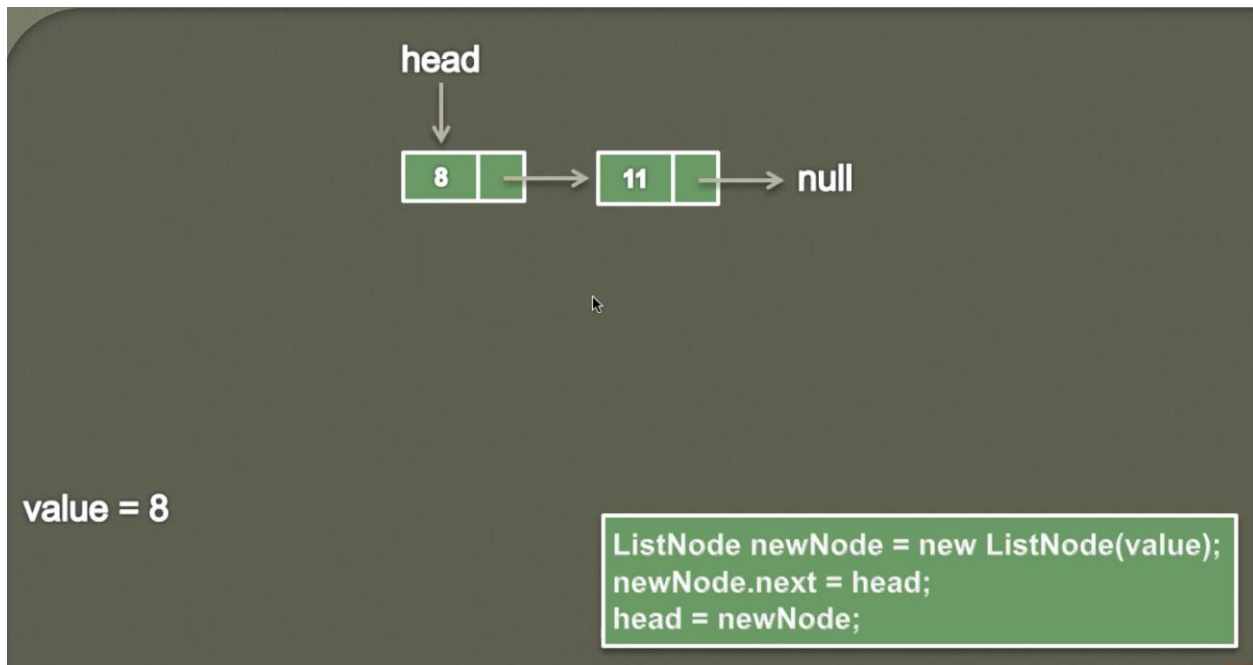
5th July 2024 – should start from videos number – 42-

https://www.youtube.com/watch?v=q99_b1z715M&list=PL6Zs6LgrJj3tDXv8a_elC6eT_4R5gfX4d&index=42

Insert new node in given position in linked list :



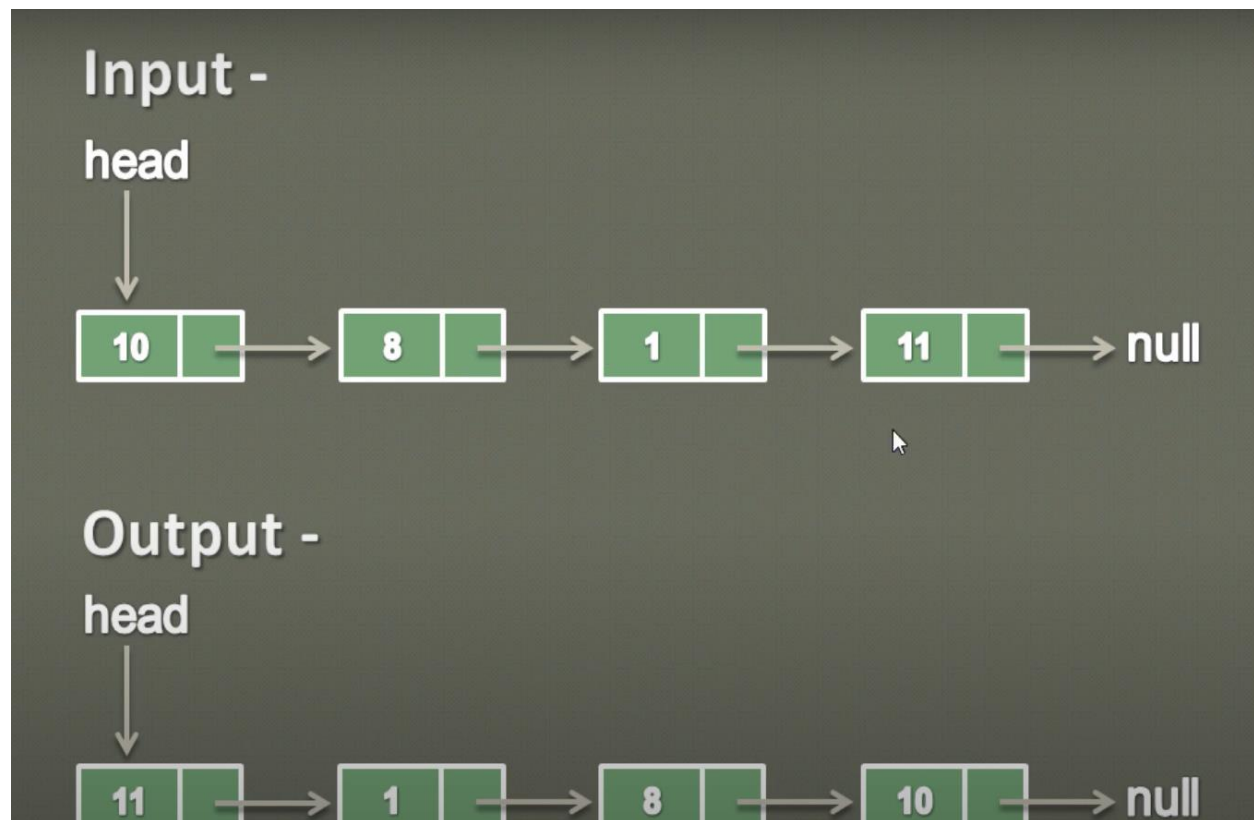
Insert node at the beginning of a Singly Linked List in Java



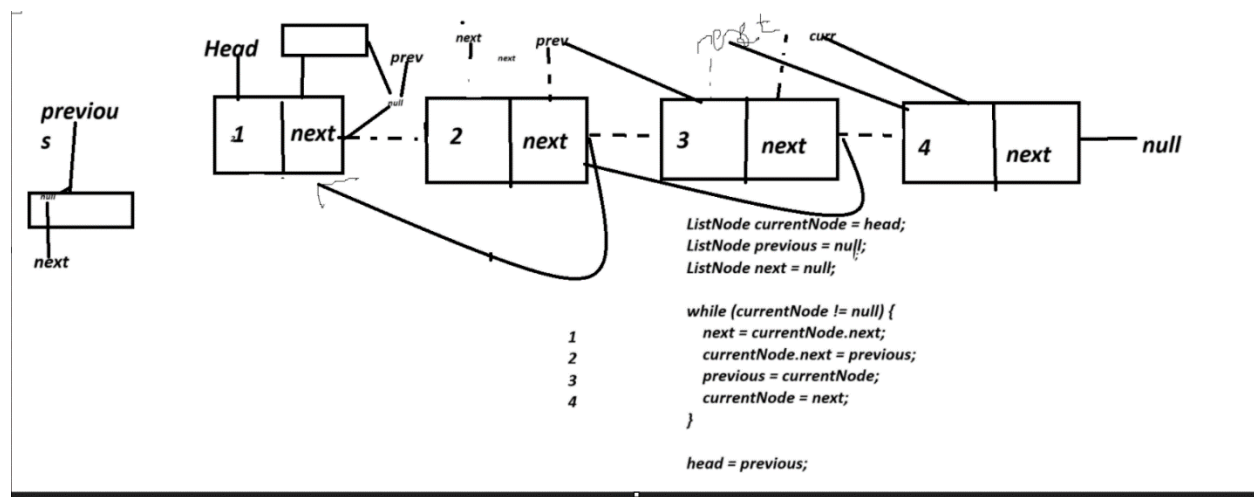
16th July 2024:

Video number – 57 - https://www.youtube.com/watch?v=jY-EUKXYT20&list=PL6Zs6LgrJj3tDXv8a_eIC6eT_4R5gfX4d&index=57

Reverse a Single Linked List in Java:

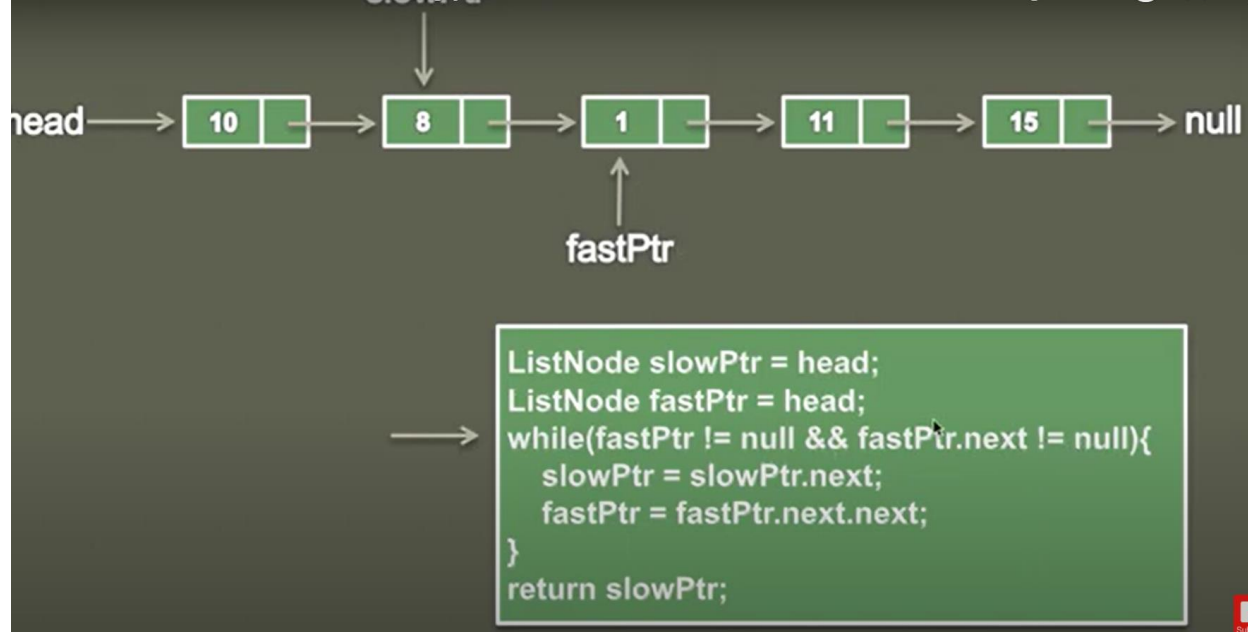


17th July 2024:



18th July 2024 :

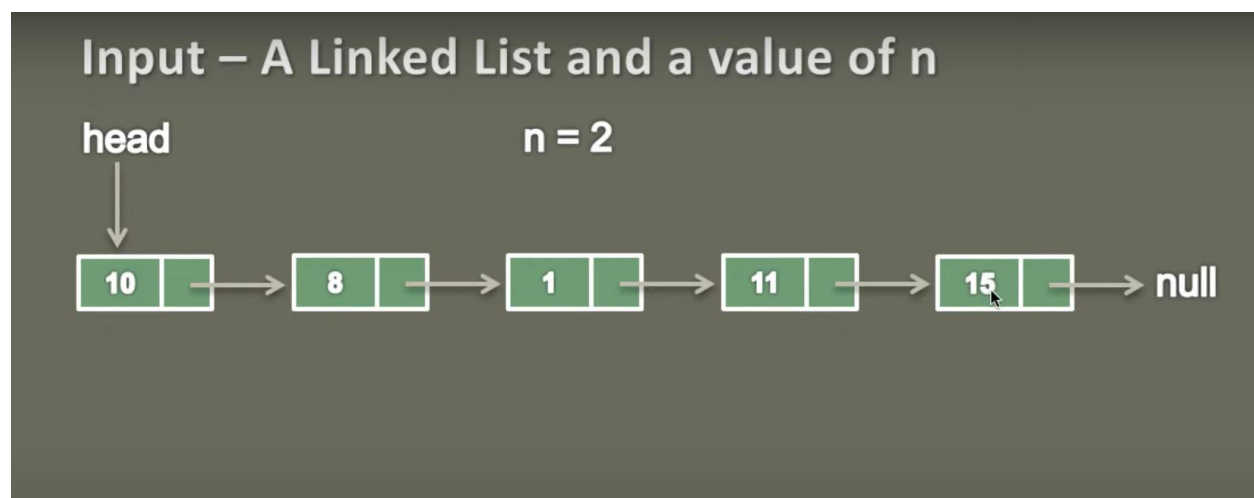
How to find middle node in a Singly Linked List in Java? | Data Structures and Algorithms

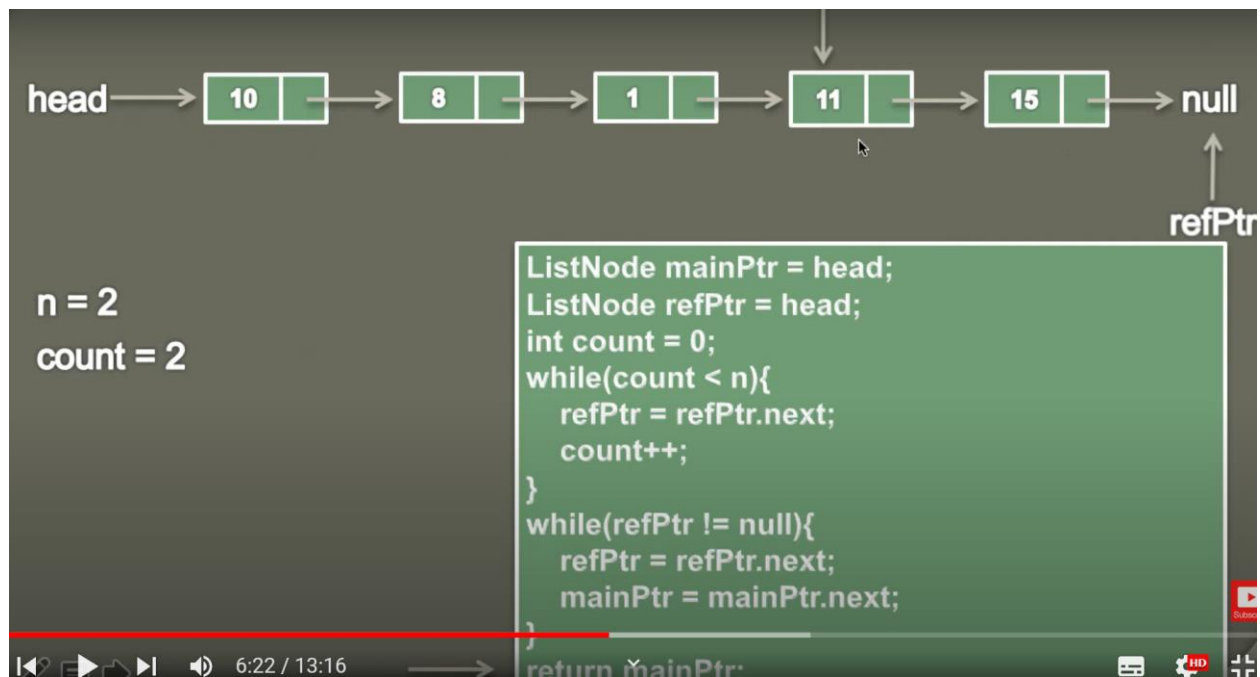


19th July 2024:-

https://www.youtube.com/watch?v=adS-s2Stg_A&list=PL6Zs6LgrJj3tDXv8a_elC6eT_4R5gfX4d&index=59

How to find nth node from the end of a Singly Linked List in Java





22nd July 2024: -

https://www.youtube.com/watch?v=0Hn18rUkZKY&list=PL6Zs6LgrJj3tDXv8a_elC6eT_4R5gfX4d&index=60

How to remove duplicate from sorted Singly Linked List in Java?

[1,2,2,2,3,3,4,5,5,6,6]

Head – 1

Tail – 6

1 ->2->2->2->3

ListNode mainPtr = head;

ListNode refPtr = head;

While(refPtr != null) {

refPtr = refPtr.next;

while(refPtr.data == mainPtr.data) {

refPtr = refPtr.next;

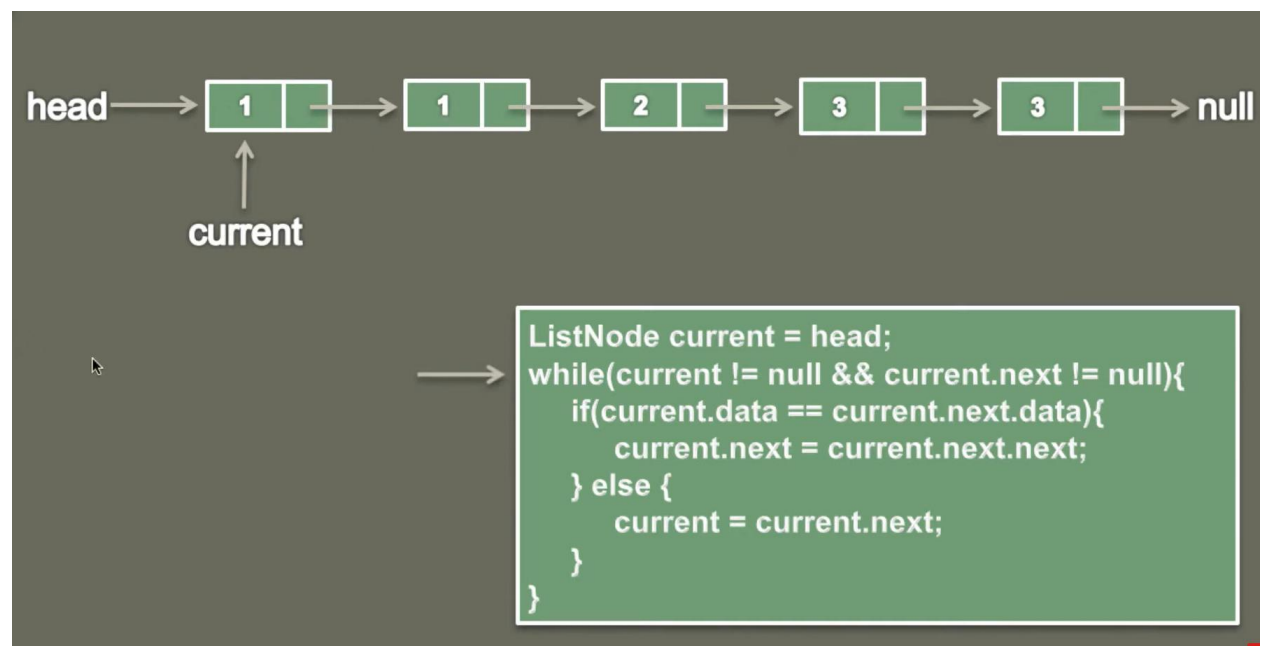
}

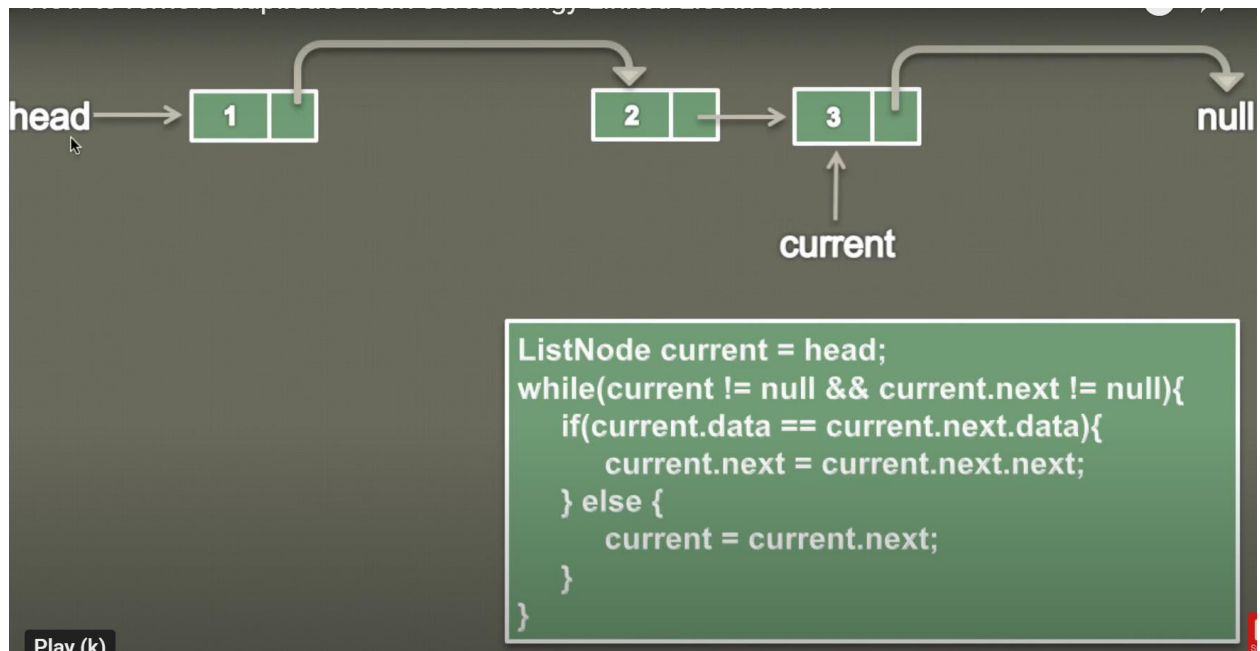
mainPtr.next = refPtr ;

mainPtr = refPtr;

}

}





24th July 2024: -

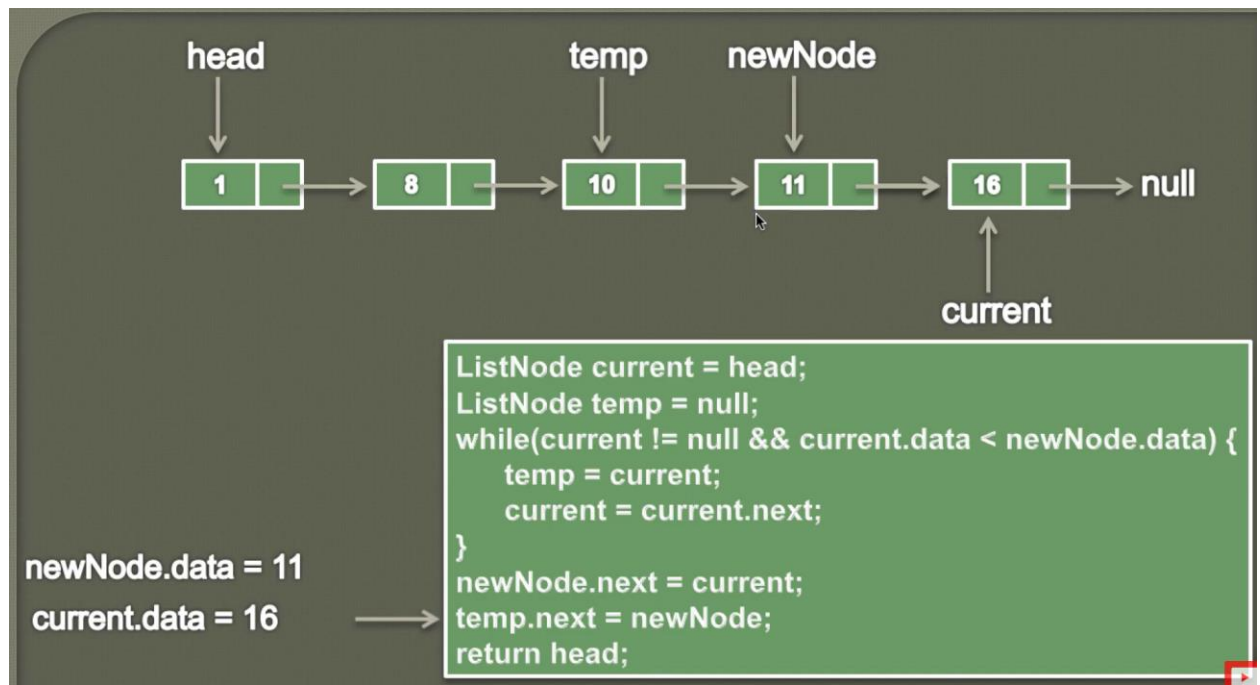
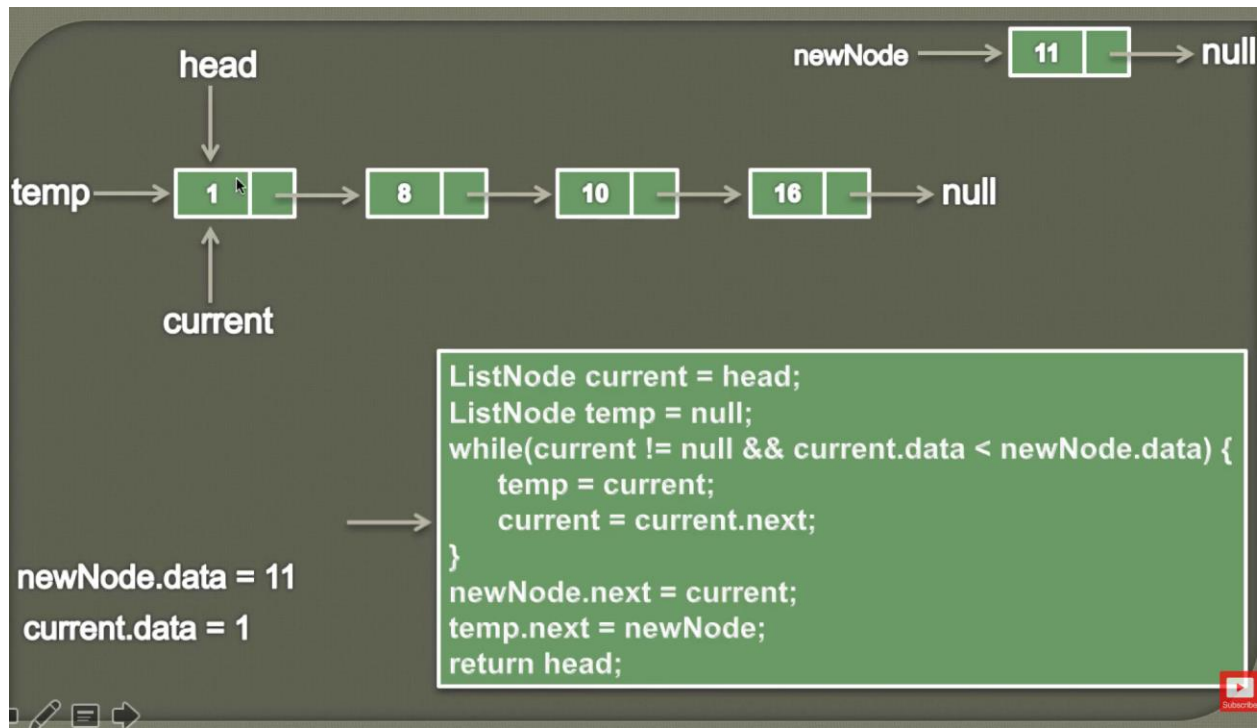
https://www.youtube.com/watch?v=u9igWutVc4o&list=PL6Zs6LgrJj3tDXv8a_elC6eT_4R5gfX4d&index=61

How to insert a node in a sorted Singly Linked List in Java ?

[1,2,3,5,6,7,8]

4-> ----->[1,2,3,4,5,6,7,8]

Current node = head;



How to remove a given key from Singly Linked List in Java?

https://www.youtube.com/watch?v=rOpnLs0lXy0&list=PL6Zs6LgrJj3tDXv8a_eIC6eT_4R5gfX4d&index=63

How to detect a loop in a Singly Linked List in Java? (Animation)

https://www.youtube.com/watch?v=rOpnLs0lXy0&list=PL6Zs6LgrJj3tDXv8a_eIC6eT_4R5gfX4d&index=63